

Protocol Design and Empirical Characterisation of a Data Channel over the Find My network

Nguyen Thai Binh

Apple's Find My network relays Bluetooth Low Energy advertisements through hundreds of millions of finder devices, and prior work has shown that arbitrary data can be smuggled through it by encoding that data into the advertised key material. That work established feasibility, not behaviour.

This report contributes `sendmy`, a two-layer protocol that separates payload encoding from transmission and derives every advertised key from a 32-byte pre-shared secret, keeping both the payload and the relayed location reports confidential; and `espbench`, an unattended test bench for measuring a shared, drifting, crowd-sourced medium.

Overnight runs characterise the result as a pure erasure channel: 99.5–99.94% deliverability at full transmit power, with no payload corruption ever observed. Reliability is dominated by broadcasts per key rather than by transmit power or rotation period, and latency is relay-bound, heavy-tailed, and measured in minutes. Deliverability collapses periodically at advertising intervals near multiples of 300 ms — an effect explained by the interval locking onto the scanners' shared listening cycle, and one that repetition cannot rescue.

August 15, 2026

Contents

1. Introduction	3
1.1. Motivation	3
1.2. Prior Works	3
1.3. Contributions	3
2. Background	4
2.1. Find My Protocol	4
2.2. NIST P-224 Elliptic Curve	4
2.3. BLE Legacy Advertising	4
3. The sendmy Data Protocol	6
3.1. Link Layer: Emitting a Valid Advertisement	6
3.2. Carrier Layer: Encoding an Arbitrary Byte	7
3.3. Reference Applications	7
4. Characterising the Channel: The espbench Test Bench	9
4.1. Questions	9
4.2. Architecture	9
4.3. Experimental Controls	10
5. Results	11
5.1. Erasure Without Corruption	11
5.2. Repetition Versus Power	11
5.3. Rotation and Broadcast-Rate Invariance	12
5.4. The 300 ms Deliverability Nulls	12
5.5. Relay-Bound Latency	14
5.6. The Rate–Reliability Trade-Off	16
6. Discussion and Interpretation	17
7. Limitations and Future Work	18
8. Conclusion	19
Use of Artificial Intelligence (AI)	20
Bibliography	21

1. Introduction

1.1. Motivation

Apple currently operates one of the largest crowd-sourced Bluetooth Low Energy (BLE) location-relay networks, powered by hundreds of millions of Find My-enabled devices [1]. In parallel, there exists a large class of low-power embedded devices carrying only a 2.4 GHz radio for BLE communication. Such a device can advertise to nearby listeners but cannot, on its own, reach the wider internet.

Its mechanics — finders relaying valid BLE advertisements to the backend — are exactly those of a one-way message-passing service that allows a BLE-only device to piggyback off the network and send data to the Internet without any infrastructure of the sender’s own.

1.2. Prior Works

That this is possible at all has been shown before. Prior proof-of-concept work demonstrated that arbitrary bytes can be smuggled through such a network by encoding them into the advertised key material and querying the backend for their reappearance [2], and open reimplementations of the offline-finding cryptography made the mechanism inspectable and reproducible [3]. What that work established is feasibility. This report expands on that work and analyses the **behaviour** instead: how *reliable* the channel is, how *quickly* a message arrives, which transmitter parameters actually matter, and *how much data per second* can be pushed through.

1.3. Contributions

My contributions are as follows.

1. **sendmy** — a two-layer protocol that, unlike previous works, decouples encoding from transmission and keeps payloads confidential end to end under a 32-byte pre-shared key (Section 3);
2. **espbench** — an unattended test bench and host harness designed around the statistical hazards of measuring a shared, drifting, crowd-sourced medium over an unreliable one-way network;
3. Empirical characterisation of the channel: a near-lossless erasure channel whose reliability is dominated by broadcasts per key, whose latency is relay-bound and heavily right-tailed, and which suffers periodic deliverability nulls at advertising intervals near multiples of 300 ms (Section 5 and Section 6); and finally,
4. Practical transmit rules for network building and a methodological warning about adaptive-timeout measurement of heavy-tailed processes (Section 6 and Section 7).

2. Background

2.1. Find My Protocol

Apple’s **Find My** network - the part relevant here is sometimes called **Offline Finding (OF)** - lets a lost device be located even when it has no Internet connection of its own. The mechanism works roughly as follows [4]:

1. A lost device continuously broadcasts a BLE advertisement containing a rotating public key.
2. Any nearby Apple device (“finder”) that hears the advertisement encrypts its own current location to that public key and uploads the encrypted report to Apple’s servers.
3. The owner of the lost device, who knows the corresponding private key, later downloads and decrypts the reports to recover the device’s location history.

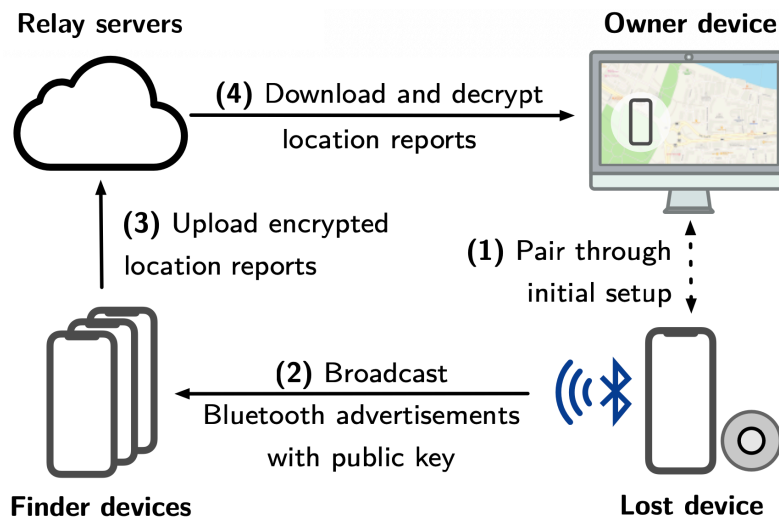


Figure 1: Simplified OF workflow [4]

Two properties of this design make it attractive to repurpose. First, the finder relays beacons *opportunistically and anonymously* - it does not authenticate the lost device [4]. Second, because the location is encrypted to a key the finder cannot read, Apple’s network is content-agnostic about what it carries: the advertised “public key” is just an opaque blob that happens to be a valid curve point.

2.2. NIST P-224 Elliptic Curve

Due to the limited payload size, Apple opts to use the NIST P-224 elliptic curve as its cryptographic primitive [4], which has a compressed public key size of 29 bytes¹. In theory, this fits within the 33-byte payload limit of a legacy BLE advertisement (see Section 2.3). However, the Find My protocol also carries metadata such as battery level, so it strips the most significant byte from the compressed public key before packing it in, leaving a payload of 28 bytes [4].

2.3. BLE Legacy Advertising

The Find My protocol — and therefore the channel built on it — rests entirely on the legacy (non-connectable) advertising mode of BLE, as defined in the core specification [6]: the device periodically transmits an advertising packet that any scanner in range may receive, with no

¹The compressed public key of P-224 ECC consists of the 28-byte x -coordinate of the public point prepended with one additional byte for the sign y -coordinate (`0x02` for positive, `0x03` for negative) [5].

connection, association, or acknowledgement. Figure 2 shows the structure of a BLE advertising packet.

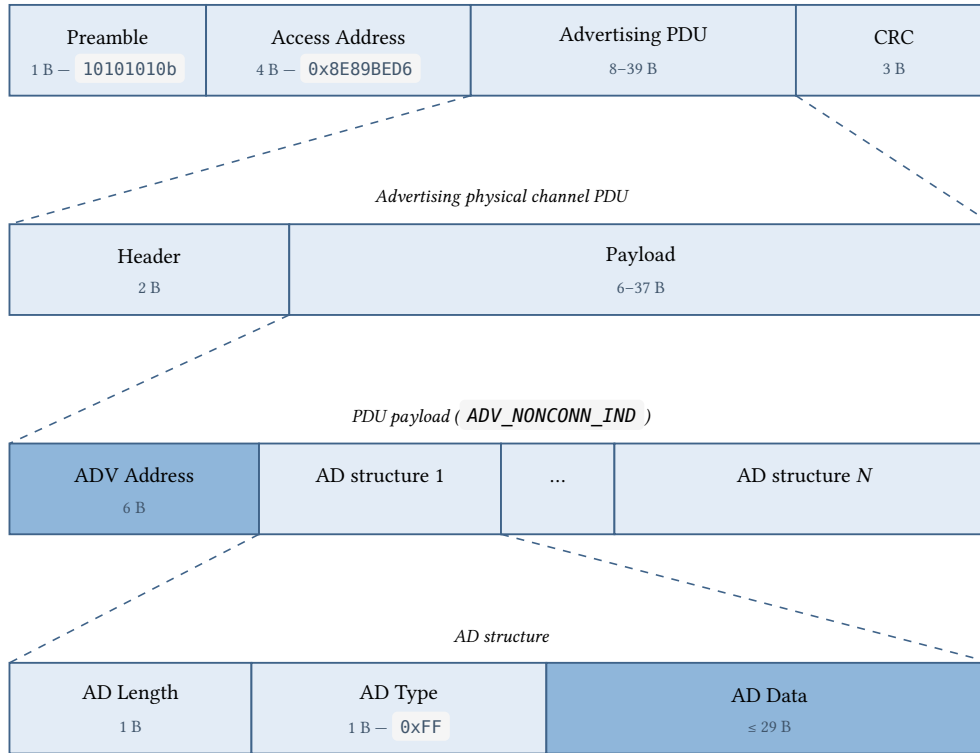


Figure 2: BLE advertising packet structure, drawn after the core specification [6]. The shaded ADV Address and AD Data fields are what we can freely set; the AD structures together occupy at most 31 B.

Since the advertising data (AD) field can contain at most 31 bytes (only 27 of them usable), together with the 6-byte device address a BLE advertisement may encode at most 33 bytes². Table 1 shows how Apple splits the 28-byte key in two: the first 6 bytes become the device address, and the remaining 22 — with the displaced top two bits of the first byte — are packed with metadata into the AD field.

NAME	SIZE (B)	DESCRIPTION/VALUE
AD length	1	0x1F (31 bytes)
AD type	1	0xFF (Manufacturer-specific data)
Company ID	2	0x004C (Apple)
OF type	1	0x12
OF length	1	0x19 (25 bytes)
Status	1	0x00 (Metadata)
Key[6..27]	22	Last 22 bytes of the 28-byte key
Key[0] >> 6	1	Top 2 bits of Key[0]
Hint	1	0x00 (Unknown purpose)

Table 1: Find My BLE advertising payload format [4]. The first 6 bytes of the key are used as the BLE device address.

²The two most significant bits of the device address must be set to 0b11 to indicate it is a static random address, as mandated by the BLE specification [6]. This is also shown in Table 1.

3. The `sendmy` Data Protocol

`sendmy` is organised into two layers. The **link layer** places an arbitrary 28-byte key onto the air as a structurally valid Find My advertisement. The **carrier layer** sits above it and decides *which* key to advertise, so that a one-byte payload can be recovered by a receiver who never sees the transmitter directly. The two reference applications sit above both.

3.1. Link Layer: Emitting a Valid Advertisement

The relay network files reports against the advertised public key, so the addressable unit of the channel is a single 28-byte key. The link layer has one job: to serialise that key into a well-formed advertising event, and to rotate keys safely. It scatters the 28 bytes across the two settable regions of Table 1, while the framing bytes reproduce the structure finder devices expect, so that to any scanner the result is an ordinary Find My advertisement. The layer is thread-safe against the advertising task, so the layer above can rotate the advertised key at any moment without racing the radio.

Figure 3 confirms this on the air. An nRF Sniffer capture of a transmitting XIAO ESP32-S3 is dissected by Wireshark without any custom decoder: the packet is an ordinary `ADV_NONCONN_IND` whose advertising data is manufacturer-specific, carries Apple's company ID `0x004C`, and nothing in the frame distinguishes it from a genuine locator tag. The byte-level breakdown below it accounts for all 31 advertising bytes and matches Table 1 field for field, with the 28-byte carrier split across the device address and the AD structure exactly as the format requires.

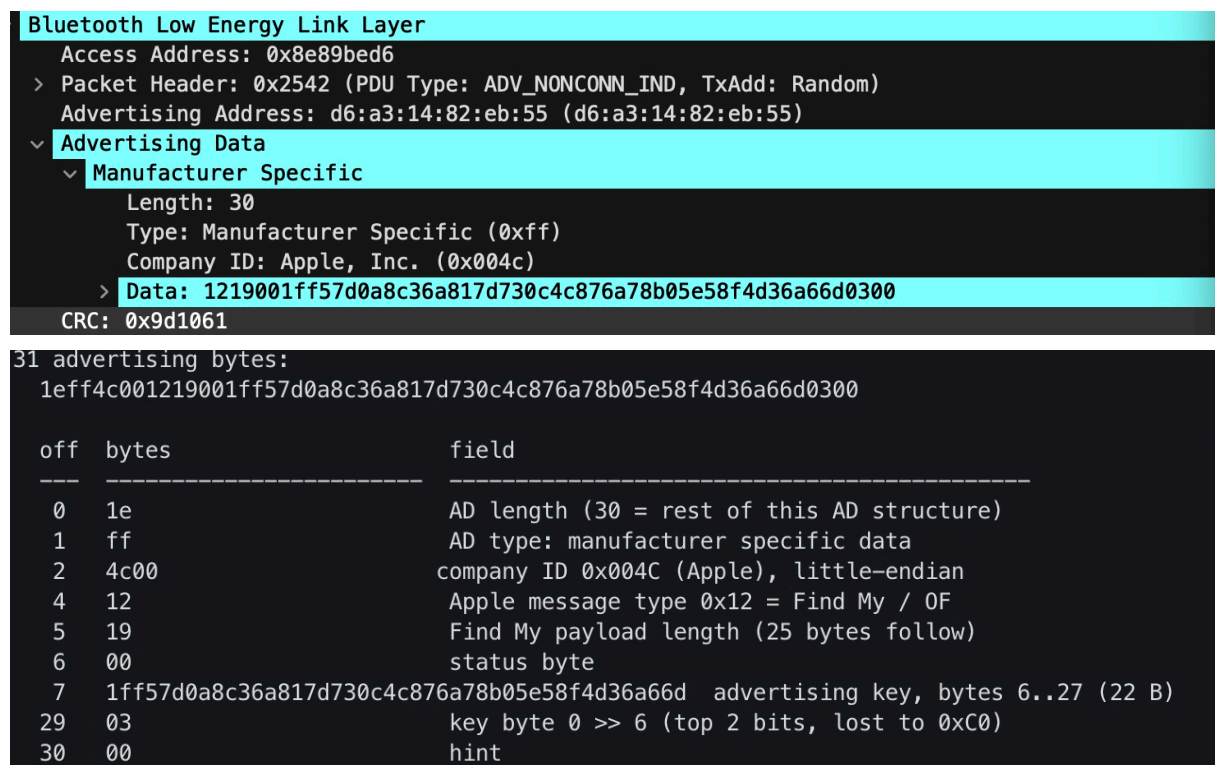


Figure 3: The link layer as seen from outside. **Top:** Wireshark's stock dissection of a captured advertisement, resolving it to manufacturer-specific data under Apple's company ID `0x004C`.

Bottom: the same 31 advertising bytes mapped field by field, matching Table 1.

3.2. Carrier Layer: Encoding an Arbitrary Byte

The link layer attaches no meaning to the key it advertises. Assigning meaning is the carrier layer's job: it turns a one-byte payload into a 28-byte key that a receiver holding the protocol, the message-identifier schedule, and a shared secret can invert. The **message identifier** `mid` is a 32-bit counter naming each message on the channel; there is no synchronisation, as the sender simply advances it monotonically and the receiver follows the same schedule.

The shared secret is a **32-byte pre-shared key**, which I call the `uid`. It is generated once, installed on both sender and receiver, and is the only secret the two share. Encoding then proceeds in three steps.

1. The `uid` is used directly as the pseudorandom key of an HKDF-Expand step [7], itself built on HMAC [8]. The Extract stage is skipped, since a random 32-byte `uid` is already a uniform key.
2. The expansion runs over the info string `"smv1" || mid_be32 || payload || attempt`, where `mid_be32` is the big-endian message identifier, `payload` is the byte being sent, and `attempt` is a rejection-sampling counter.
3. The 28-byte output block is read as a big-endian integer and taken as a scalar d . The transmitted carrier is the big-endian x -coordinate of the point $d \cdot G$ on NIST P-224 / secp224r1 [5], where G is the standard base point.

A raw block may equal or exceed the group order, in which case `attempt` is incremented and the expansion repeated. Rejection sampling therefore guarantees that the carrier is always a genuine point on the curve — and that guarantee is what makes the channel a clean **erasure channel**. Since every carrier the transmitter can emit is valid by construction, a receiver's failure to find a report cannot be an encoding fault, but lack of nearby finder devices.

Recovery is an **existence oracle** over the relay backend. Because the payload is a single byte, a given `mid` has only 256 possible carriers. The receiver derives all 256 locally under the shared `uid` and asks the backend which of them has a report on file; the one that does reveals the transmitted byte.

The `uid` is also what makes the channel private end to end. Without it — for a BLE eavesdropper, a protocol-aware observer, or the backend operator — the 256 candidates are not computable, since the candidate space is then the whole curve rather than 256 points, and the private scalar d needed to decrypt the finder-side location reports cannot be derived. Both the payload and the location reports are therefore readable only by holders of the `uid`.

3.3. Reference Applications

Two applications in the `sendmy` repository^o exercise the protocol: `esptag`^o, a rotating locator-tag beacon standing in for a generic Find My device, and `espsend`^o, a temperature sender that transmits one signed byte per message identifier. Both are built on ESP-IDF [9] and the NimBLE host stack [10], and run on a Seeed XIAO ESP32-S3 — a compact module representative of the BLE-only devices the channel is meant to serve. Figure 4 shows the bench setup.

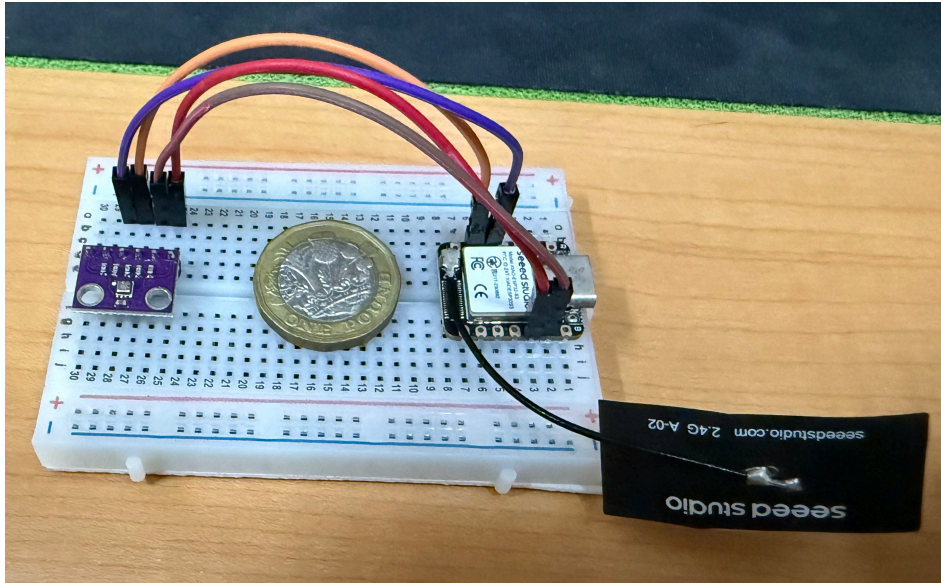


Figure 4: The transmitter: a Seeed XIAO ESP32-S3 with its external 2.4 GHz antenna (right), and the BME280 that `espsend` reads over I2C (left). A one-pound coin gives the scale.

Running `esptag` end to end exercises every layer at once. The beacon rotates through its key schedule on air; the receiver, holding only the `uid`, derives the same schedule locally and queries the backend for each candidate. Figure 5 shows the result of one such run: of 100 epoch keys derived from the shared secret, 18 had reports on file, each decrypting to a location fix with an accuracy estimate. No key that was never broadcast ever returned a report.

```
> scripts/.venv/bin/python scripts/fetch_reports.py --epochs 100
Derived 100 epoch keys (0..99); p_0 = 7735d57a9fc2728202812fcd32a6b1b268773642a020b2bdc4574b3.
Restored session for [redacted]

18 report(s) across 100 epoch(s) (oldest first):
[epoch 0] 2026-06-13T11:38 [redacted] lat=+1.292 [redacted] lon=+103.772 [redacted] ±60m conf=0
[epoch 0] 2026-06-13T11:39 [redacted] lat=+1.292 [redacted] lon=+103.771 [redacted] ±28m conf=0
[epoch 0] 2026-06-13T11:39 [redacted] lat=+1.292 [redacted] lon=+103.772 [redacted] ±56m conf=0
[epoch 1] 2026-06-13T11:40 [redacted] lat=+1.292 [redacted] lon=+103.772 [redacted] ±40m conf=0
[epoch 1] 2026-06-13T11:40 [redacted] lat=+1.292 [redacted] lon=+103.772 [redacted] ±74m conf=0
[epoch 1] 2026-06-13T11:40 [redacted] lat=+1.292 [redacted] lon=+103.772 [redacted] ±74m conf=0
```

Figure 5: Recovering an `esptag` transmission. The receiver derives 100 epoch keys from the `uid` and fetches the reports filed against them; each line is one decrypted location. Identifiers and precise coordinates are redacted.

4. Characterising the Channel: The `espbench` Test Bench

The medium is a live, shared, drifting, crowd-sourced network that cannot be instrumented from the inside, so the measurement apparatus is itself a substantial part of the work.

4.1. Questions

A transmitter is characterised by the triple (A, T, B) together with its transmit power. The **advertising interval** A is the nominal time between successive advertising events; the **key rotation period** T is how long a single key is broadcast before rotation; and the **broadcasts per key** $B = T/A$ follows from the other two, so fixing any two of the three fixes the third. Transmit power is compared at a low (-24 dBm) and a high ($+9$ dBm) setting throughout.

The test bench is built to answer four questions:

- **Deliverability**: what fraction of transmitted keys are ultimately retrievable?
- **Latency**: how long after a broadcast does a report become fetchable, and is the delay in discovery or in propagation to the backend?
- **Which parameter matters**: of A, T, B , and the transmit power, which move deliverability and which are inert?
- **Goodput**: how many payload bytes per second of on-air time can the channel carry, and at what reliability?

4.2. Architecture

The firmware is flashed once and thereafter reconfigured over UART, so the host harness sets A, T , the transmit power, and the key schedule at runtime without reflashing. This is what makes an unattended overnight sweep possible.

The **cell** is the experimental unit of everything that follows: one combination of transmitter parameters, sent as roughly twenty consecutive keys within a single contiguous time window, under its own freshly minted 32-byte uid and its own disjoint range of message identifiers. The last two give independence across cells on a shared relay, making it most unlikely that two cells ever collide on the same backend key.

Two mechanisms observe delivery. A two-tier **live poller** watches for reports as the run proceeds, giving a real-time view of progress. Then — and this is the authoritative measurement — an **offline resweep** re-fetches every key after the fact, with no queue cap and unbounded patience. It is the resweep, never the live poller, that defines deliverability ground truth.

Ambient finder density was logged throughout as a covariate, and Figure 6 shows it: a day-night cycle falling to roughly 9–13 finder devices in the evening trough and rising to about 22 at the early-morning peak. This variation modulates the channel, but it does not explain the deliverability effects, which persist across the whole cycle.

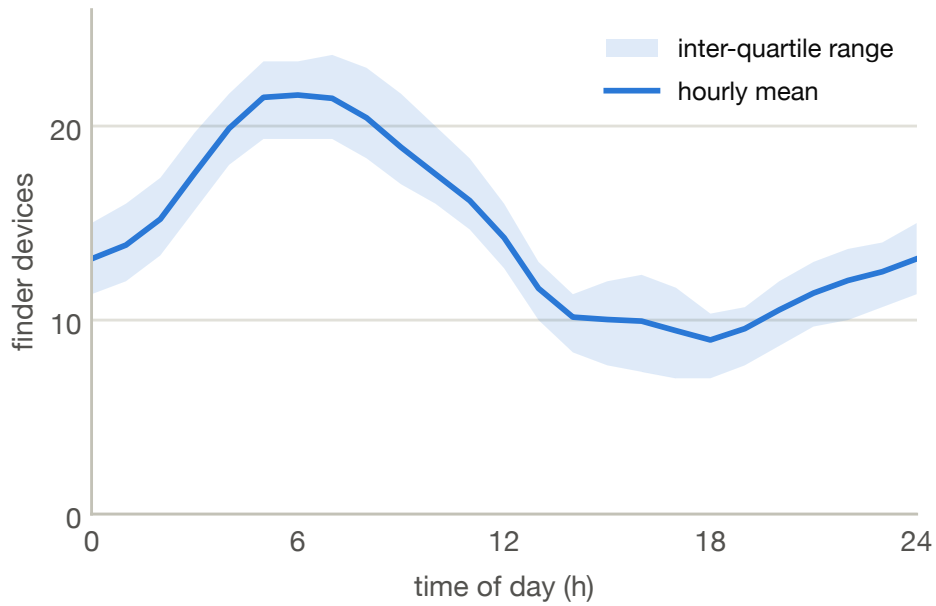


Figure 6: Ambient finder density over the course of a run, sampled continuously as a covariate. Density follows a day-night cycle, with an afternoon and evening trough and an early-morning peak.

4.3. Experimental Controls

Three controls defend the measurements against the peculiarities of an overnight, crowd-sourced medium.

- **Round-robin interleaving.** Conditions are interleaved rather than run in blocks, so that the slow day-night drift in finder density is spread evenly across all cells, instead of confounding whichever condition happened to run at 3 a.m.
- **Anchor cells.** A fixed reference condition is repeated throughout, making up roughly 7–10% of each run, as an instrument check: if the anchors misbehave, the night is suspect.
- **Cell-clustered testing.** The keys within a cell share a uid and a time window, so they are not independent samples. Significance is therefore assessed by permutation tests that resample at the level of the cell, not the key.

The anchor condition is chosen for stability rather than sensitivity, so the anchors can flag a bad night but cannot calibrate a difference **between** nights. Accordingly, I treat only **within-run** contrasts as effects, and never compare absolute rates across runs.

5. Results

Throughout, absolute rates are quoted within a run and are not compared across runs.

5.1. Erasure Without Corruption

At full transmit power and urban finder density, per-run deliverability sits between about 99.5% and 99.94%, and across every run and every cell **no payload corruption was ever observed**. This follows directly from the carrier construction of Section 3.2: a key either reappears intact or does not reappear at all. The channel is thus a **pure erasure channel**, and the receiver always knows when a loss has occurred, because the oracle query for that message identifier returns nothing.

5.2. Repetition Versus Power

The single most effective control on deliverability is the number of broadcasts per key, B . Figure 7 shows misses decaying roughly geometrically in B : at -24 dBm deliverability climbs through 42.8, 66.1, 83.3, 97.2, and 100% as B goes 1, 2, 4, 8, 16, while at $+9$ dBm it starts high at $B = 1$ (86.1%) and saturates quickly. This is the shape of independent Bernoulli chances to be heard, each broadcast being another opportunity for some finder device to catch the key; in the low-power arm a single per-broadcast miss probability of about 0.6 reproduces the whole sweep to within four points. Transmit power does not change the shape, only **where** the curve saturates. The practical reading is simple: B is the knob to turn for reliability, and power decides how few broadcasts suffice.

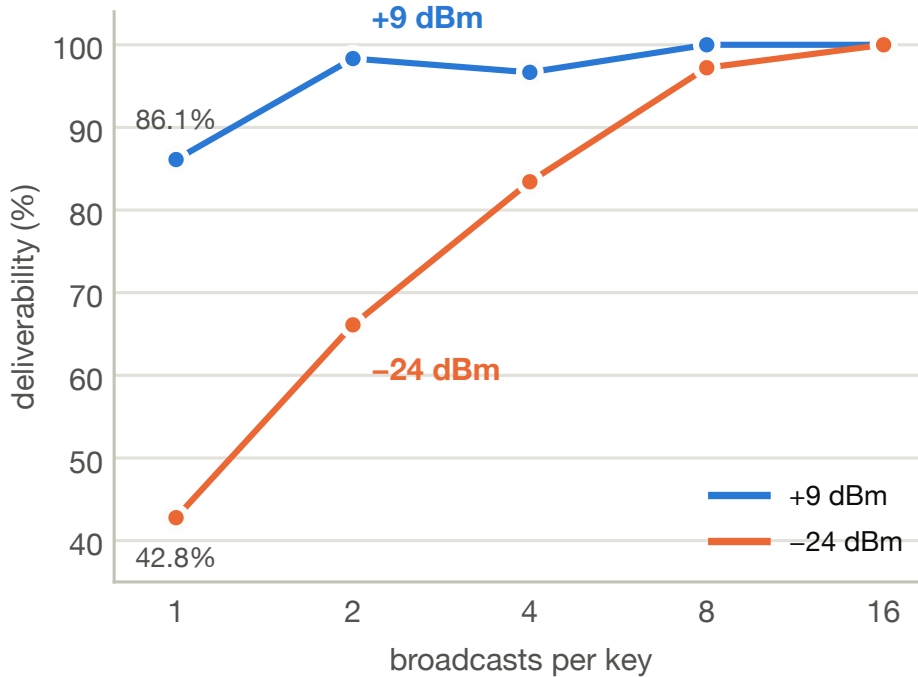


Figure 7: Deliverability against broadcasts per key $B \in \{1, 2, 4, 8, 16\}$ at -24 dBm and $+9$ dBm. Misses decay approximately geometrically in B .

5.3. Rotation and Broadcast-Rate Invariance

At full power the channel sits at its deliverability ceiling, and there the key rotation period T and the raw broadcast rate are inert: varying them produces no resolvable change, because there is no headroom above the ceiling for them to occupy. This is not a claim that T never matters — it matters for latency (Section 5.5) — but at full power it is hidden beneath the ceiling.

5.4. The 300 ms Deliverability Nulls

The most surprising finding concerns the advertising interval A . One might expect deliverability to vary smoothly with A ; instead it shows sharp, regularly spaced deficits, which I call **periodic deliverability nulls**. Figure 8 sweeps A from 200 to 1400 ms at -24 dBm and shows a pronounced dip at every multiple of 300 ms: the intervals $A \in \{300, 600, 900, 1200\}$ ms average 69.3% deliverability against 92.7% at the surrounding intervals, a gap a cell-clustered permutation test places well beyond chance ($p < 5 \times 10^{-5}$, the floor of a 20,000-shuffle test). The run pins B at two levels, 4 and 6, so that a penalty tracking A must show up at the **same** intervals in both arms, whereas one tracking T would show up at **different** ones. It shows up at the same intervals.

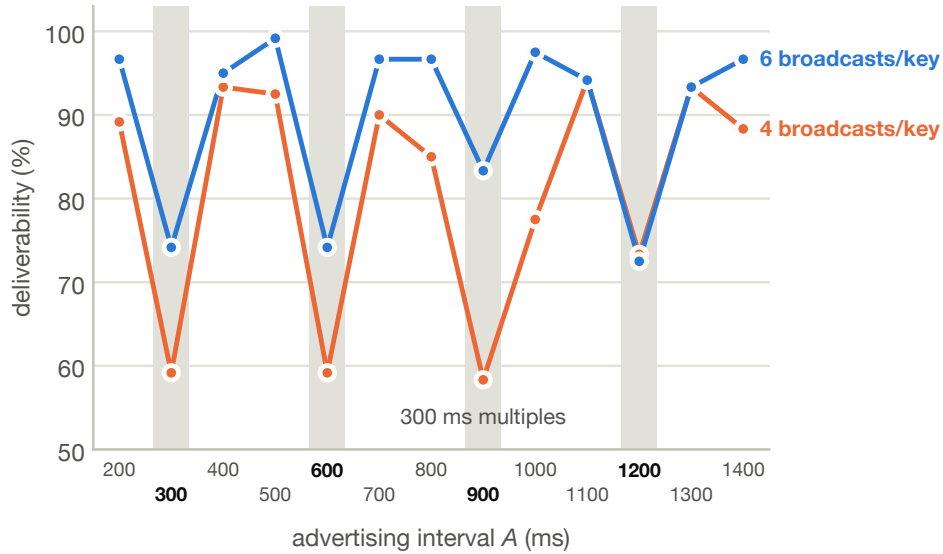


Figure 8: Deliverability against advertising interval A from 200 to 1400 ms at -24 dBm, at 4 and 6 broadcasts per key. Sharp periodic deliverability nulls appear at every multiple of 300 ms.

The explanation is plain arithmetic. A scanner does not listen continuously: under the core specification [6] it opens a **scan window** once per fixed **scan interval** and is deaf for the rest of each interval. If a coordinated fraction of finder devices open their windows on a fixed period near 300 ms, a transmitter whose interval is an integer multiple of that period keeps landing at the same position within the scan cycle; should that position fall in the deaf gap, every broadcast misses in exactly the same way. An interval that is **not** a multiple of 300 ms instead drifts across the cycle, sampling both open and closed windows. Figure 9 sketches this geometry.

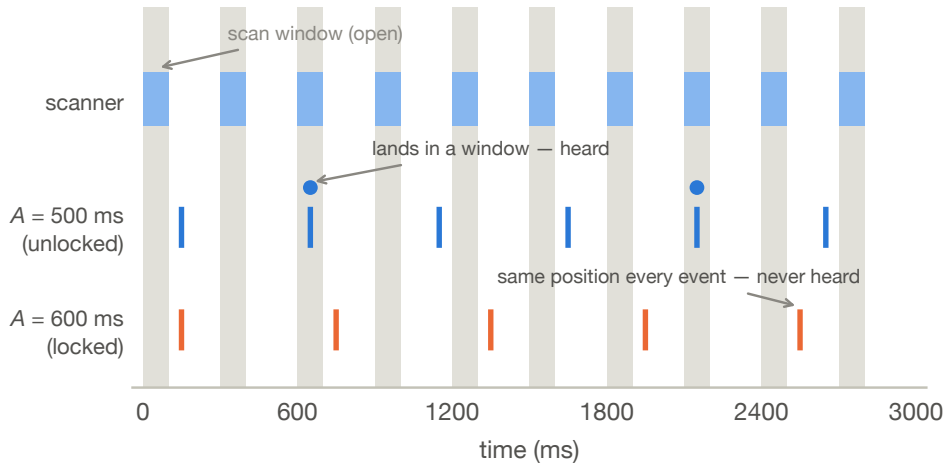


Figure 9: Schematic of the scan-cycle geometry (window width and starting offset are illustrative, not measured). The scanner opens a window once per 300 ms cycle. An advertiser at $A = 600$ ms revisits the same position on every event and, having started in the deaf gap, is never heard; an advertiser at $A = 500$ ms drifts across the cycle and is heard whenever an event lands inside a window.

Figure 10 makes this precise, plotting deliverability against the number of distinct positions in the scan cycle a given interval reaches — a count driven by $\gcd(A, 300\text{ms})$ — at a matched four broadcasts per key. An interval stuck at a single position delivers 48.3%; deliverability then climbs monotonically to 85.0% at six positions.

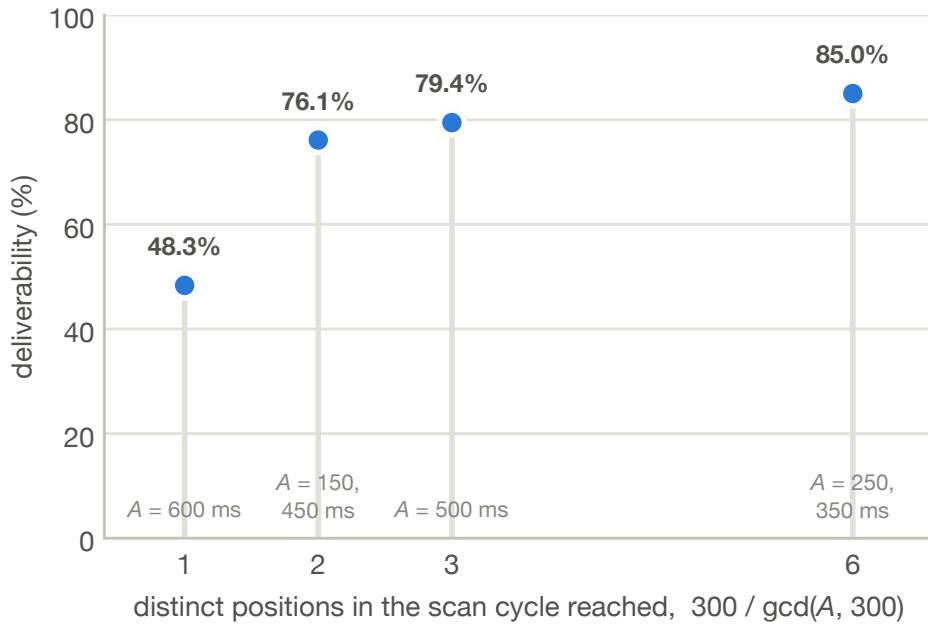


Figure 10: Deliverability against the number of distinct positions in the scan cycle the advertiser reaches, an argument driven by $\gcd(A, 300\text{ms})$.

A natural hope is that an interval stuck at one position could be rescued simply by broadcasting each key more often. It cannot. Figure 11 contrasts a control interval $A = 500$ ms (three positions) with the locked interval $A = 600$ ms (one position) as B rises to 16: the unlocked interval reaches 98.9%, while the locked one stalls at 76.7%. Repeated broadcasts at a locked

interval are **synchronised repeats** — every one lands at the same unfavourable position, so they are not independent opportunities. The null can only be escaped by choosing the interval.

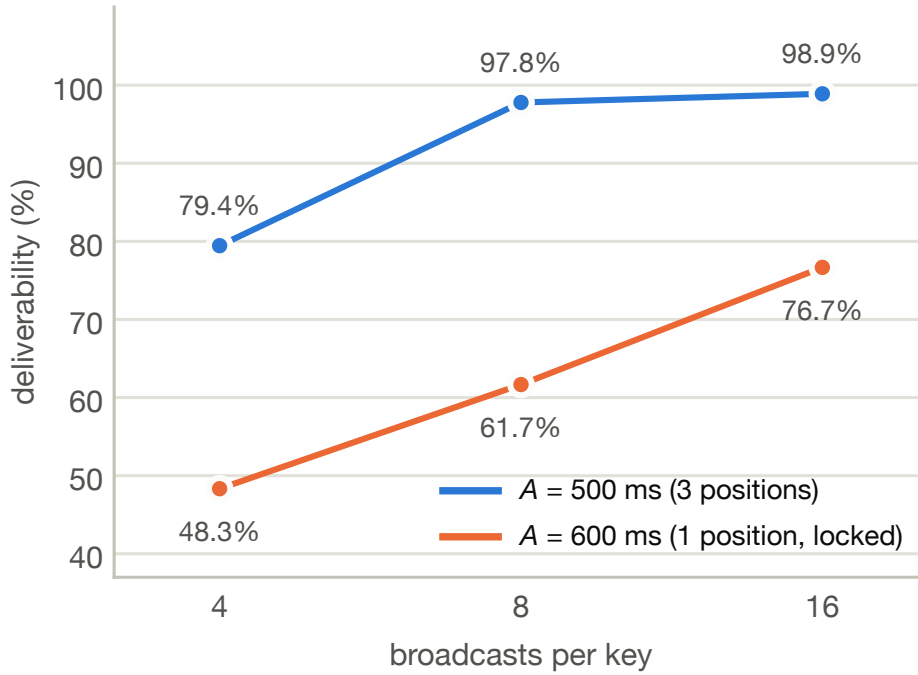


Figure 11: Deliverability as B increases to 16 for a control interval $A = 500$ ms (three positions in the scan cycle) and a locked interval $A = 600$ ms (one position).

5.5. Relay-Bound Latency

Delivery, when it happens, is not prompt. Figure 12 gives the cumulative distribution of propagation latency, from a key’s first broadcast to its first fetchable report, at full transmit power: the median is 158 s and the ninetieth percentile 356 s, with a pronounced heavy tail.

Both figures are lower bounds, and deliberately so. Propagation is timed by the live poller, which gives up on a key once its 600 s of patience are spent, so an abandoned key drops out of the curve rather than being recorded as slow. The 15% of keys it never timed are, by construction, the slow ones, so the true distribution lies to the right of the one plotted.

The delay is not in discovery. Live detections show that a finder device typically hears the advertisement almost immediately; the lag is overwhelmingly the time for the finder to upload its report and for the backend to make it fetchable. The channel is genuinely store-and-forward, and it is the store that dominates.

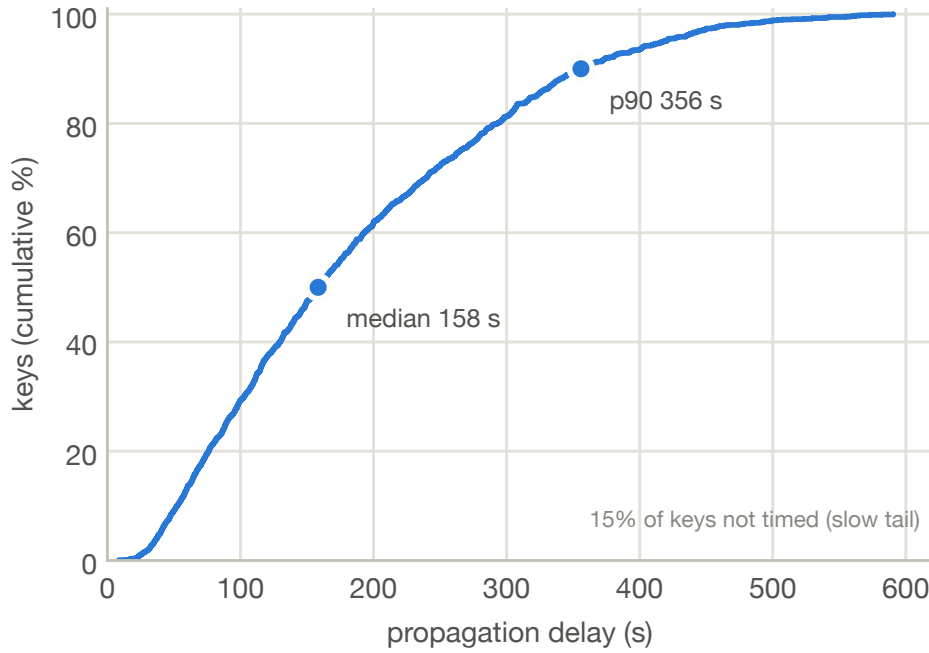


Figure 12: Cumulative distribution of propagation latency, from a key’s broadcast to its first fetchable report, over the factorial run that pins broadcasts per key independently of the rotation period, at full transmit power. The plotted percentiles are lower bounds set by the live poller’s patience.

Latency is also signal-dependent, and this is the one place where the rotation period is not inert. Figure 13 plots median propagation latency against key rotation period and broadcast count. Twenty broadcasts per key rather than five buys about 40 s off the median (138 s against 178 s), because the earliest broadcast to be relayed sets the clock and more broadcasts give more early chances. Stronger signal helps too: the median was 308 s at +9 dBm against 405 s at −24 dBm. In every case the distribution stays heavy-tailed — the median moves, but the tail persists.

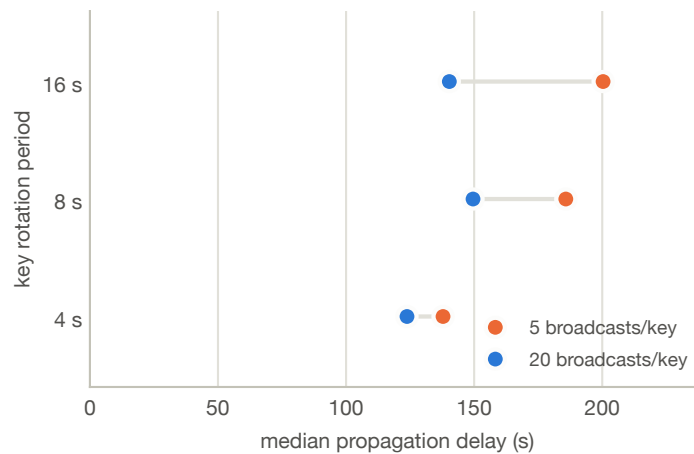


Figure 13: Median propagation latency against key rotation period and broadcast count, from the factorial run that pins broadcasts per key independently of the rotation period.

5.6. The Rate–Reliability Trade-Off

I define goodput as delivered payload bytes per second of wall-clock on-air time. Because each key carries exactly one payload byte, goodput is simply the delivered-key rate, counted from the authoritative offline resweep. There is no single goodput figure, only a goodput–reliability trade-off. In the full-power, high-reliability regime (rotation periods of 4, 8, and 16 s, at roughly 100% deliverability), goodput ranges from 0.062 to 0.246 byte/s. The fastest point I measured is 1.22 byte/s at 80.6% deliverability ($A = 150$ ms, $T = 600$ ms, -24 dBm); a more defensible operating point, balancing the two, is 0.87 byte/s at 91.7% deliverability ($A = 200$ ms, $T = 1000$ ms, -24 dBm). Full-power sub-second rotation was never measured, so a higher goodput at high reliability is plausible but remains an extrapolation.

Figure 14 puts both coordinates on one plot, for every condition of the interval sweep of Figure 8. Most conditions sit well inside the frontier, and the intervals locked to a multiple of 300 ms sit furthest inside it: a null costs rate and reliability at once, since the keys it loses are keys already paid for in on-air time. Within that run the frontier has only three corners — 89.2% at 1.04 byte/s, 96.7% at 0.77 byte/s, and 99.2% at 0.32 byte/s — so what a transmitter actually gets to choose is one of a handful of operating points, not a position on a smooth dial.

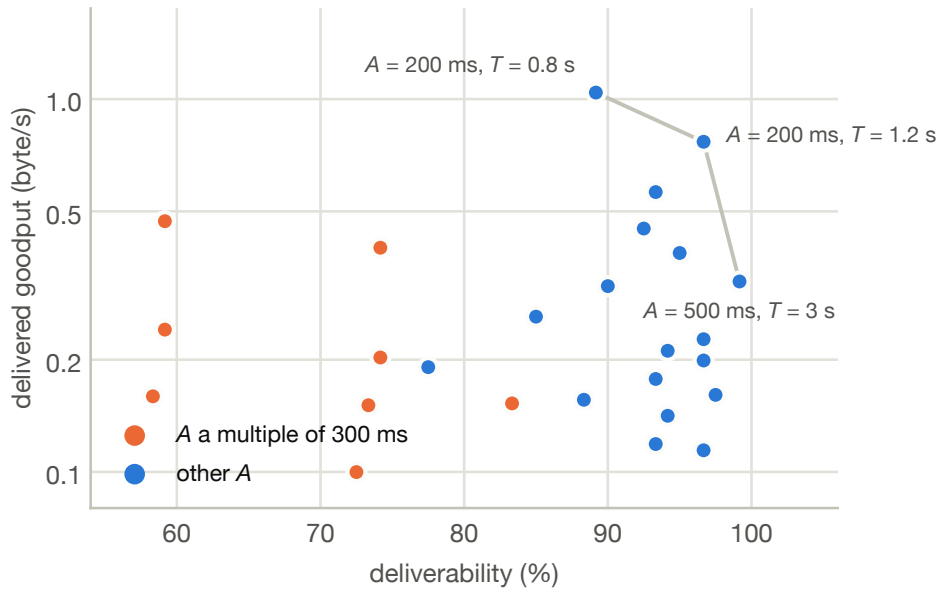


Figure 14: Delivered goodput against deliverability for all 26 conditions of the advertising-interval sweep at -24 dBm, both axes taken from the offline resweep. The line traces the Pareto frontier; note the logarithmic goodput axis.

6. Discussion and Interpretation

Taken together, the results describe a near-lossless, store-and-forward erasure channel with minutes-scale, heavy-tailed latency. It corrupts nothing, loses little at full power, and never delivers quickly. This shape has three direct consequences.

The first is practical — a small set of transmit rules.

- **Broadcast each key at least eight times ($B \geq 8$).** This puts deliverability at or near the ceiling at full power, and well up the geometric curve even at low power; it also shaves tens of seconds off the median latency.
- **Never choose an advertising interval at or near a multiple of 300 ms.** Intervals such as 250, 400, 500, and 700 ms sample several positions in the scan cycle and are safe, whereas 300, 600, 900, and 1200 ms are stuck at a single position and cannot be rescued by extra broadcasts.

The second consequence is inferential. The periodic nulls are, in effect, a measurement of the unobserved scanner population made through the channel itself: their 300 ms spacing implies that a substantial, coordinated fraction of finder devices scan on a shared periodic schedule, since a population scanning at random offsets could not produce a sharp, interval-locked null. The channel thus reveals a structural property of the crowd that cannot be observed directly.

The third consequence is methodological, and it is the one I would most want a reader to take away. An early live-poller episode showed how an adaptive-timeout observer of a heavy-tailed process can introduce a bias that is not merely noisy but **systematically correlated with the independent variable**, fabricating a clean and plausible trend where none exists. The defence is a separation of concerns: monitor progress with the adaptive instrument, but settle ground truth with a patience-unbounded resweep.

7. Limitations and Future Work

The study is deliberately deep rather than broad, and its scope bounds its claims. All the measurements come from a single site, a single board, and a single relay population, so the period at which the nulls fall, the absolute deliverability figures, and the latency distribution may all differ elsewhere. Finder density was never manipulated, only observed, so I cannot report a controlled density response. The latency figures are lower bounds set by the poller’s patience and cannot be recovered by re-analysis. The periodic nulls were characterised only at -24 dBm, where the low ceiling makes them visible; their behaviour at full power is unmeasured. And because the anchor cells cannot calibrate a between-night difference, every absolute figure is confined to within-run interpretation.

Two follow-up measurements would be especially valuable. The first is a long-broadcast **escape** experiment at the locked interval $A = 600$ ms, with B pushed as high as 128, to find the broadcast count (if any) at which deliverability finally recovers. The second is a fine notch-width sweep, in 5–10 ms steps across roughly 590–610 ms, to measure how wide the null actually is, and hence how much margin the “avoid multiples of 300 ms” rule really needs. Beyond measurement, the protocol invites extension: multi-byte framing, forward error correction to trade goodput for resilience against the residual erasures, and hardening of the shared-secret scheme — rotating the uid for forward secrecy, and adding sender authentication, since today any holder of the uid can both read and forge on the channel.

8. Conclusion

Prior work showed that Apple’s Find My network can be made to carry arbitrary bytes. This report set out to establish what that channel actually is.

The answer is a pure erasure channel with minutes-scale latency. At full transmit power it delivers almost everything — 99.5–99.94% per run — and corrupts nothing. Reliability is governed chiefly by the number of broadcasts per key, with transmit power setting only where the curve saturates. The one sharp failure mode is a family of periodic deliverability nulls at advertising intervals near multiples of 300 ms, which lock the advertiser to a fixed position in the scanners’ shared listening cycle and which no amount of extra broadcasting can escape. The cost is latency: delivery is relay-bound and heavy-tailed, with a median of minutes, and goodput runs between 0.06 and 1.2 byte/s depending on how much reliability one is willing to trade away.

Reaching those numbers took as much design as measurement. `sendmy` contributes the keyed carrier construction that makes every emitted key a valid curve point, so that a miss is unambiguously network loss. `espbench` contributes the apparatus, and with it a warning I would repeat to anyone measuring a heavy-tailed process: an adaptive-timeout observer can manufacture a trend that is not there, and only a patience-unbounded resweep settles ground truth.

The channel is therefore highly reliable but latency-bound — well suited to asynchronous, delay-tolerant messaging from BLE-only devices, and unsuited to anything real-time.

Use of Artificial Intelligence (AI)

AI has been used in various areas of this project, in accordance with the University's Policy for Use of AI in Teaching and Learning. This section describes the aspects and extents to which AI was used.

AI is **never** used in the following areas:

- Ideation of the `sendmy` protocol;
- Secondary research;
- Interpretation of experiment results;
- Prose writing of the final report; and
- This section on the "Use of AI".

Furthermore, AI is used **sparingly** in the following areas:

- Checking for misspellings and grammar errors in the final report;
- Implementation of the `sendmy` protocol on ESP32; and
- Implementation of the `espbench` project.

Finally, AI is used **substantially** in the following areas:

- Refactoring of P-224 cryptography of MicroECC library [11] on ESP32;
- Automated execution of testing matrices for `espbench` ;
- Raw data analysis;
- Design of the poster; and
- Conversion of raw Markdown file into Typst-coded source file.

Regardless of the extents of AI usage, I remain responsible for all project deliverables - the poster and this report - as well as any code written.

Bibliography

- [1] Apple Inc., ‘Apple Platform Security’. Accessed: Oct. 10, 2020. [Online]. Available: <https://support.apple.com/guide/security/>[◦]
- [2] F. Bräunlein, ‘Send My: Arbitrary Data Transmission via Apple’s Find My Network’. [Online]. Available: <https://positive.security/blog/send-my>[◦]
- [3] A. Heinrich and M. Stute, *OpenHaystack: Build Your Own Find My Network*. (Mar. 2021). [Online]. Available: <https://github.com/seemoo-lab/openhaystack>[◦]
- [4] A. Heinrich, M. Stute, T. Kornhuber, and M. Hollick, ‘Who Can Find My Devices? Security and Privacy of Apple’s Crowd-Sourced Bluetooth Location Tracking System’, *Proceedings on Privacy Enhancing Technologies (PoPETs)*, vol. 2021, no. 3, pp. 227–245, Mar. 2021, doi: 10.2478/popets-2021-0045[◦].
- [5] L. Chen, D. Moody, A. Regenscheid, A. Robinson, and K. Randall, ‘SP 800-186: Recommendations for Discrete Logarithm-based Cryptography — Elliptic Curve Domain Parameters’, Feb. 2023. doi: 10.6028/NIST.SP.800-186[◦].
- [6] Bluetooth SIG, ‘Bluetooth Core Specification, Version 5.4’, Jan. 2023. [Online]. Available: <https://www.bluetooth.com/specifications/specs/core-specification/>[◦]
- [7] H. Krawczyk and P. Eronen, ‘RFC 5869: HMAC-based Extract-and-Expand Key Derivation Function (HKDF)’, May 2010. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc5869>[◦]
- [8] H. Krawczyk, M. Bellare, and R. Canetti, ‘RFC 2104: HMAC: Keyed-Hashing for Message Authentication’, Feb. 1997. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc2104>[◦]
- [9] Espressif Systems, ‘ESP-IDF Programming Guide’. Accessed: Aug. 09, 2026. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>[◦]
- [10] Espressif Systems, *esp-nimble: Espressif Port of the Apache Mynewt NimBLE Bluetooth Low Energy Stack*. (Mar. 2019). [Online]. Available: <https://github.com/espressif/esp-nimble>[◦]
- [11] K. MacKay, *micro-ecc: ECDH and ECDSA for 8-bit, 32-bit, and 64-bit Processors*. (Jan. 2022). [Online]. Available: <https://github.com/kmackay/micro-ecc>[◦]